

조인(JOIN)

ch06.sql 코드를 실행 순서대로 분석한 학습자료입니다. HR 스키마 (EMPLOYEES/DEPARTMENTS/JOBS) 기준입니다.

06

01 전통적 조인(WHERE) vs ANSI JOIN

두 테이블을 PK-FK 관계로 연결하는 두 가지 문법. 결과는 동일하지만 가독성과 표준 준수 여부가 다릅니다.

```
-- 전통적 조인: FROM에 테이블 나열 + WHERE에 연결조건
select first_name, e.department_id, d.department_name
from employees e, departments d
where e.department_id = d.department_id;

-- ANSI JOIN: JOIN ~ ON으로 연결조건을 명시적으로 분리
select first_name, e.department_id, d.department_name
from employees join departments d
    on e.department_id = d.department_id;
```

권장 — ANSI JOIN은 조인조건(ON)과 필터조건(WHERE)이 분리되어 가독성이 좋고, 실수로 조인 조건을 빠뜨려 카티션 곱(cartesian product)이 발생할 위험도 적습니다. 실무 표준입니다.

02 3-way JOIN — 세 테이블 연결

Sales 부서(DEPARTMENTS) 소속 직원(EMPLOYEES)의 직책명(JOBS)을 구하는 예제 — 3개 테이블을 두 번의 조인으로 연결합니다.

```
-- ANSI 방식
select e.first_name, j.job_title
from employees e join departments d on e.department_id = d.department_id
      join jobs j on e.job_id = j.job_id
where d.department_name = 'Sales';

-- 전통적 방식(WHERE에 조인조건 2개 + 필터조건 1개, 총 3개 AND)
select e.first_name, j.job_title
from employees e, departments d, jobs j
where d.department_name = 'Sales'
      and e.department_id = d.department_id
      and e.job_id = j.job_id;
```

테이블이 N개면 조인조건은 N-1개 — 3개 테이블을 연결하려면 조인조건(ON 또는 WHERE의 연결식)이 최소 2개 필요합니다. 하나라도 빠지면 의도치 않은 카티션 곱이 발생합니다.

03 SELF JOIN — 같은 테이블을 자기 자신과 조인

EMPLOYEES 테이블 안에서 **manager_id** 가 다시 **employee_id** 를 가리키는 자기참조 구조를 풀어 내려면, 같은 테이블에 별칭을 두 개 부여해 서로 다른 테이블처럼 조인합니다.

```
-- 직원별 담당 매니저 이름 조회
select a.employee_id, a.first_name, a.manager_id, b.first_name
from employees a join employees b on a.manager_id = b.employee_id
order by a.employee_id;
```

읽는 법 — a는 "본인", b는 "그 사람의 매니저" 역할입니다. a.manager_id = b.employee_id 조건으로, a의 manager_id 값과 일치하는 employee_id를 가진 b행(=매니저의 정보)을 찾아 붙입니다.

주의 — 최상위 관리자(manager_id가 NULL)는 이 조인 결과에서 **제외**됩니다(내부조인이므로). 전원을 포함하려면 LEFT OUTER JOIN이 필요합니다.

두 SELECT 결과를 세로로 합치거나 비교합니다. 두 쿼리의 컬럼 개수와 자료형이 일치해야 합니다.

```
select department_id from employees
UNION    -- 합집합, 중복 제거 + 정렬됨
select department_id from departments;

select department_id from employees
UNION ALL -- 합집합, 중복 유지(정렬 X, 더 빠름)
select department_id from departments;

select department_id from employees
INTERSECT -- 교집합: 양쪽에 공통으로 존재하는 값만
select department_id from departments;

select department_id from departments -- 27개 부서
minus
select distinct department_id from employees -- 차집합: 앞 결과 - 뒤 결과
where department_id is not null; -- 결과: 직원이 한 명도 없는 부서 목록(11개)
```

연산자	의미	중복
UNION	합집합	제거
UNION ALL	합집합	유지
INTERSECT	교집합	제거
MINUS	차집합(앞-뒤)	제거

MINUS는 순서에 민감 — A MINUS B와 B MINUS A는 결과가 다릅니다. 예제는 "부서는 있지만 소속 직원이 없는 부서"를 구하는 전형적인 패턴입니다.